

1. Prove convexity of logistic loss.

Logistic loss (also called **log loss** or **binary cross-entropy loss**) is a convex loss function used in **binary classification**, especially in **logistic regression**.

It measures how well a model's predicted probability matches the true class label.

Consider binary classification.

$x \in \mathbb{R}^d$: feature vector

$y \in \{-1, +1\}$: class label

$w \in \mathbb{R}^d$: weight vector (model parameters)

$w^T x$: dot product of weights and features

$z = yw^T x$: margin (signed distance)

The logistic loss for one data point is

$$\ell(w) = \log(1 + e^{-yw^T x})$$

Here,

$\log$ is the natural logarithm,

e is the base of the natural exponential function.

Let

$$z = yw^T x$$

Then the loss becomes

$$\ell(z) = \log(1 + e^{-z})$$

First derivative with respect to z :

$$\ell'(z) = \frac{-e^{-z}}{1 + e^{-z}}$$

This can be written as

$$\ell'(z) = -\frac{1}{1 + e^z}$$

Second derivative:

$$\ell''(z) = \frac{e^z}{(1 + e^z)^2}$$

Since

$e^z > 0$ for all real z ,

$(1 + e^z)^2 > 0$,

we get

$$\ell''(z) > 0$$

Therefore, the logistic loss is convex in z .

Now consider convexity with respect to w .

$$z = yw^T x$$

This is a linear function of w .

Property: If a function $f(z)$ is convex and z is an affine (linear) function of w , then the composition $f(z(w))$ is convex in w .

Since logistic loss is convex in z , and z is linear in w ,

$$\ell(w) = \log(1 + e^{-yw^T x})$$

is convex in w .

Hessian form:

Gradient:

$$\nabla \ell(w) = -\frac{yx}{1 + e^{yw^T x}}$$

Hessian:

$$\nabla^2 \ell(w) = \frac{e^{yw^T x}}{(1 + e^{yw^T x})^2} xx^T$$

Here,

$\nabla \ell(w)$ is the gradient vector,

$\nabla^2 \ell(w)$ is the Hessian matrix,

xx^T is the outer product matrix.

Since the scalar term is positive and xx^T is positive semi-definite, the Hessian is positive semi-definite.

Therefore, the logistic loss function

$$\ell(w) = \log(1 + e^{-yw^T x})$$

is convex.

2. Why does logistic regression not overfit as easily as high-degree polynomial regression?

1. Model Complexity

Logistic regression is a linear model with limited flexibility,

whereas high-degree polynomial regression creates very complex curves that can closely fit training noise, increasing the risk of overfitting.

2. Decision Boundary Simplicity

Logistic regression produces a linear decision boundary (or mildly nonlinear with selected features), while high-degree polynomials create highly wiggly boundaries that adapt excessively to small fluctuations in data.

3. Number of Parameters

Logistic regression typically uses fewer parameters, whereas polynomial regression with high degree introduces many additional parameters, increasing variance and overfitting risk.

4. Smooth Loss Function

Logistic regression uses a convex logistic loss function that penalizes large errors smoothly, while polynomial regression (using squared error) can fit extreme values aggressively.

5. Built-in Regularization (Common Practice)

Logistic regression is commonly trained with L1 or L2 regularization, which controls coefficient magnitude, whereas high-degree polynomial regression without regularization easily memorizes training data.

6. Bias–Variance Tradeoff

Logistic regression has relatively higher bias and lower variance, making it more stable, while high-degree polynomial regression has low bias but very high variance, leading to overfitting.

3. Derive IRLS from Newton's method.

IRLS (Iteratively Reweighted Least Squares) is an optimization algorithm used to estimate parameters in models where ordinary least squares cannot be directly applied, especially in logistic regression and generalized linear models (GLMs). In logistic regression, the relationship between input features and output probabilities is nonlinear (due to the sigmoid function). So we cannot solve it using ordinary least squares directly.

IRLS converts the nonlinear optimization problem into a sequence of weighted least squares problems that are easier to solve.

The Hessian matrix is a square matrix of second-order partial derivatives of a scalar function. It describes the curvature of the function.

Here is the clean derivation **without intermediate horizontal lines**, in simple step-by-step format with all terms defined clearly.

Derivation of IRLS from Newton's Method

We derive Iteratively Reweighted Least Squares (IRLS) for logistic regression using Newton's method.

Definitions

$x_i \in \mathbb{R}^d$: feature vector for the i -th sample

$y_i \in \{0, 1\}$: class label

$w \in \mathbb{R}^d$: parameter (weight) vector

$X \in \mathbb{R}^{n \times d}$: design matrix containing all feature vectors

n : number of samples

Sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Predicted probability

$$p_i = \sigma(x_i^T w)$$

Vector form

$$p = \sigma(Xw)$$

Log-likelihood of logistic regression

$$\ell(w) = \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

We maximize this function.

Newton's Method

Newton's update rule is

$$w^{new} = w^{old} - H^{-1}g$$

where

$g = \nabla \ell(w)$ is the gradient vector

$H = \nabla^2 \ell(w)$ is the Hessian matrix

Gradient of log-likelihood

$$g = X^T(y - p)$$

Here

y is the vector of true labels

p is the vector of predicted probabilities

Hessian matrix

$$H = -X^T W X$$

where W is a diagonal matrix with entries

$$W_{ii} = p_i(1 - p_i)$$

This term represents the variance of each Bernoulli observation.

Substitute gradient and Hessian into Newton's update

$$w^{new} = w^{old} - (-X^T W X)^{-1} X^T (y - p)$$

Since inverse of a negative matrix introduces a minus sign

$$w^{new} = w^{old} + (X^T W X)^{-1} X^T (y - p)$$

Now define the adjusted response variable

$$z = X w^{old} + W^{-1}(y - p)$$

Rewriting the update

$$w^{new} = (X^T W X)^{-1} X^T W z$$

This is exactly the solution of a weighted least squares problem.

IRLS Algorithm Steps

1. Compute $p_i = \sigma(x_i^T w)$
2. Compute weights $W_{ii} = p_i(1 - p_i)$
3. Compute adjusted response $z = Xw + W^{-1}(y - p)$
4. Update

$$w = (X^T W X)^{-1} X^T W z$$

Why it is called IRLS

Iteratively means the procedure is repeated until convergence.

Rewighted means weights $W_{ii} = p_i(1 - p_i)$ change in every iteration.

Least Squares means each step solves a weighted least squares problem.

Conclusion

IRLS is obtained by applying Newton's method to maximize the logistic regression log-likelihood. Each Newton update becomes a weighted least squares solution, which gives the IRLS algorithm.

4. Compare logistic loss vs hinge loss mathematically.

Binary classification setting:

$x \in \mathbb{R}^d$: feature vector

$y \in \{-1, +1\}$: true class label

$w \in \mathbb{R}^d$: weight vector

$w^T x$: linear score

$z = yw^T x$: margin

Margin z measures how confidently a point is classified.

Logistic Loss

Formula:

$$L_{log}(z) = \log(1 + e^{-z})$$

Definition:

Logistic loss is a smooth convex loss function used in logistic regression. It penalizes misclassification and also penalizes correct classifications with low confidence.

Hinge Loss

Formula:

$$L_{hinge}(z) = \max(0, 1 - z)$$

Definition:

Hinge loss is a convex loss function used in Support Vector Machines (SVM). It penalizes points that are misclassified or lie inside the margin boundary.

- Logistic loss is smooth, probabilistic, and always penalizes errors gradually.

- Hinge loss is piecewise linear, margin-based, and ignores well-classified points beyond the margin.

5. What happens when data is perfectly separable?

Perfectly separable data means there exists a weight vector w such that

$$y_i w^T x_i > 0 \text{ for all } i$$

This means one straight line (or hyperplane) separates the two classes without any error.

What happens in Logistic Regression:

1. No finite solution

Logistic regression maximizes log-likelihood. If data is perfectly separable, the likelihood keeps increasing as weights increase. So there is no finite optimal value of w .

2. Weights go to infinity

The coefficients grow very large in the separating direction. Mathematically, $\|w\| \rightarrow \infty$.

3. Probabilities become 0 or 1

Predicted probability

$$P(y = 1|x) = \frac{1}{1 + e^{-w^T x}}$$

As $w^T x \rightarrow \infty$, probability $\rightarrow 1$

As $w^T x \rightarrow -\infty$, probability $\rightarrow 0$

4. Algorithm may not converge

Since the optimal solution is at infinity, gradient descent or Newton's method may not properly converge.

5. Regularization fixes the problem

If we add a penalty term like

$$\lambda \|w\|^2$$

then the weights remain finite and a unique solution exists.

Final idea

When data is perfectly separable, logistic regression tries to become infinitely confident, causing weights to grow without bound unless regularization is added.

6. Why does MLE not exist under perfect separation?

Maximum Likelihood Estimation is a method used to estimate model parameters by maximizing the likelihood function.

In logistic regression, MLE chooses w that maximizes:

$$\ell(w) = \sum_{i=1}^n \log \left(1 + e^{-y_i w^T x_i} \right)$$

where

x_i = feature vector

$y_i \in \{-1, +1\}$ = class label

w = parameter vector

MLE exists if there is a **finite value of w** that maximizes this function.

Perfect separation means there exists a vector w such that

$$y_i w^T x_i > 0 \quad \text{for all } i$$

Reason 1: Likelihood keeps increasing

If data is perfectly separable, we can multiply w by a large constant c .

$$y_i (c w^T x_i) \rightarrow +\infty$$

Then

$$\log(1 + e^{-y_i c w^T x_i}) \rightarrow 0$$

So the log-likelihood keeps improving as $c \rightarrow \infty$.

Reason 2: No maximum at finite value

Since increasing $\|w\|$ always increases likelihood, there is no finite value where the likelihood is maximized.

Reason 3: Parameters diverge

The optimal solution occurs only when

$$\|w\| \rightarrow \infty$$

Thus, the MLE does not exist in finite form.

- MLE requires a finite parameter value that maximizes likelihood.
- Under perfect separation, the likelihood increases indefinitely as weights grow larger.
- Therefore, no finite maximum exists, and MLE does not exist

7. Derive logistic regression as maximum entropy classifier.

$x \in \mathbb{R}^d$: feature vector

$y \in \{0, 1\}$: class label

$p(y|x)$: conditional probability

Entropy of a distribution:

$$H(p) = - \sum_y p(y|x) \log p(y|x)$$

Maximum entropy principle:

Among all probability distributions that satisfy given constraints, choose the one with the highest entropy (least additional assumptions).

Step 1: Objective

We want to find $p(y|x)$ that maximizes entropy:

$$\max - \sum_y p(y|x) \log p(y|x)$$

Step 2: Constraints

1. Probability constraint

$$\sum_y p(y|x) = 1$$

2. Feature expectation constraint

The expected value of features under the model must match empirical data:

$$\sum_y p(y|x) yx = w^T x$$

This ensures the model respects observed feature statistics.

Step 3: Use Lagrange Multipliers

Form the Lagrangian:

$$\mathcal{L} = - \sum_y p(y|x) \log p(y|x) + \lambda \left(1 - \sum_y p(y|x) \right) + \theta^T \left(\sum_y p(y|x) yx \right)$$

where

λ = multiplier for normalization constraint

θ = parameter vector for feature constraint

Step 4: Take derivative with respect to $p(y|x)$

$$\frac{\partial \mathcal{L}}{\partial p(y|x)} = -\log p(y|x) - 1 - \lambda + \theta^T yx$$

Set derivative to zero:

$$\log p(y|x) = \theta^T yx - 1 - \lambda$$

Step 5: Solve for $p(y|x)$

$$p(y|x) = \exp(\theta^T yx - 1 - \lambda)$$

Let

$$Z(x) = \sum_y \exp(\theta^T yx)$$

Then

$$p(y|x) = \frac{\exp(\theta^T yx)}{Z(x)}$$

Step 6: Binary Case Simplification

For $y \in \{0, 1\}$:

$$p(y = 1|x) = \frac{\exp(\theta^T x)}{1 + \exp(\theta^T x)}$$

$$p(y = 0|x) = \frac{1}{1 + \exp(\theta^T x)}$$

This is exactly the logistic (sigmoid) function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Thus,

$$p(y = 1|x) = \sigma(\theta^T x)$$

Final Conclusion

By maximizing entropy subject to feature expectation constraints and probability normalization, we obtain:

$$p(y|x) = \frac{\exp(\theta^T x)}{1 + \exp(\theta^T x)}$$

This is exactly the logistic regression model.

8. Analyze asymptotic distribution of estimator.

An estimator is a rule or formula used to estimate an unknown population parameter using sample data.

Example: sample mean $\bar{X}$ estimates population mean μ .

Asymptotic means “when sample size $n \rightarrow \infty$ ”.

The asymptotic distribution of an estimator describes the probability distribution that the estimator approaches as the sample size becomes very large. It tells us how the estimator behaves when we have a very large amount of data.

The asymptotic distribution of an estimator tells us:

- What distribution the estimator follows for large n
- How it behaves around the true parameter
- How its variance decreases as sample size increases

Logistic regression uses Maximum Likelihood Estimation (MLE) to estimate w .

Asymptotic Distribution of $\hat{w}$

Under regular conditions (no perfect separation, correct model, finite variance):

$$\sqrt{n}(\hat{w} - w_0) \xrightarrow{d} N(0, I^{-1}(w_0))$$

This means:

As sample size increases, the estimator becomes approximately normally distributed.

Meaning of Terms

w_0 = true parameter value

$N(0, I^{-1}(w_0))$ = multivariate normal distribution

$I(w_0)$ = Fisher Information matrix

9. Compare generative (Naive Bayes) vs discriminative (logistic).

• Generative Model – Naive Bayes

Naive Bayes is a generative algorithm because it models how the data is generated. It learns the overall data distribution by

estimating how features relate to each class. It assumes that features are conditionally independent given the class label. After learning the distribution, it can classify new data and can also handle missing data effectively. Naive Bayes works well with small datasets and is computationally efficient. However, because of its strong independence assumption, it may struggle in complex datasets where features are highly correlated.

- **Discriminative Model – Logistic Regression**

Logistic regression is a discriminative algorithm because it directly models the decision boundary between classes. Instead of learning the full data distribution, it focuses on estimating the probability of the class given the input features. It does not assume feature independence, making it more flexible and often more accurate when sufficient training data is available. Logistic regression performs very well in classification tasks but typically requires more data than Naive Bayes for stable performance.

Key Differences

- Naive Bayes models the joint distribution of features and labels; logistic regression models the conditional probability of labels given features.
- Naive Bayes focuses on understanding how data is generated; logistic regression focuses on separating classes.
- Naive Bayes handles small or incomplete datasets better; logistic regression performs better with large datasets.
- Naive Bayes is simpler and faster; logistic regression is more flexible and often achieves higher classification accuracy.

In Summary

Naive Bayes tries to understand the overall structure of the data and

then classify, while logistic regression directly learns the best boundary to distinguish between classes.

10. When does logistic regression fail?

- **Perfect separation**
If classes are perfectly separable, the coefficients tend to infinity and no finite solution exists without regularization.
- **Strong multicollinearity**
When features are highly correlated, coefficient estimates become unstable and unreliable.
- **Non-linear decision boundary**
Logistic regression assumes a linear relationship between features and log-odds. It performs poorly if the true boundary is highly nonlinear (unless features are transformed).
- **Small sample size**
With very limited data, the model estimates have high variance and become unstable.
- **Highly imbalanced data**
If one class dominates, the model may predict mostly the majority class and fail to detect the minority class.
- **Outliers in features**
Extreme values in input variables can significantly influence parameter estimates.
- **High-dimensional data ($p \gg n$)**
When the number of features is much larger than the number of samples, the model may overfit without regularization.

